

Project 1: Socket Programming

ชื่อโปรแกรม : Hopper

ชื่อ Protocol ที่ออกแบบ : FHP (File Hopping Protocol)

วัตถุประสงค์ของโปรแกรม:

โปรแกรม Hopper เป็น network application ประเภท client-server ที่พัฒนาขึ้นเพื่อให้ผู้ใช้งานสามารถโอนย้ายไฟล์ระหว่างเครื่องตัวเองกับเครื่อง server ผ่านเครือข่ายได้ โดยเลียนแบบการทำงานพื้นฐานของ File Transfer Protocol (FTP) แต่ออกแบบให้มีขนาดเล็กและเข้าใจง่ายกว่า เหมาะสำหรับใช้ศึกษาแนวคิดของการออกแบบ application-layer protocol และการเขียนโปรแกรมด้วย socket

การใช้งาน:

- ดูรายการไฟล์ทั้งหมดที่มีบน Server (LIST)
- อัปโหลดไฟล์จากเครื่อง Client ขึ้นไปเก็บไว้บน Server (UPLOAD)
- ดาวน์โหลดไฟล์จาก Server กลับมายังเครื่อง Client (DOWNLOAD)
- ลบไฟล์ที่เก็บอยู่บน Server (DELETE)

ลักษณะของโปรแกรม:

- โครงสร้างเป็นแบบ Client-Server โดย Server หนึ่งตัวสามารถให้บริการ Client ได้หลายคนพร้อมกัน (รองรับ concurrent connections ด้วย multi-threading)
- ข้อมูลที่รับส่งมี 2 ลักษณะ คือ 1) ข้อความคำสั่ง/สถานะที่เป็น text ขนาดเล็ก และ 2) เนื้อไฟล์ที่เป็น binary และอาจมีขนาดใหญ่ ต้องส่งแบบแบ่งเป็นส่วน (chunk/stream)
- มีมาตรการความถูกต้องของข้อมูลแบบสมบูรณ์ (data integrity) กล่าวคือไฟล์ที่โอนเสร็จสิ้นต้องมีเนื้อหาเหมือนต้นฉบับ ทุก byte ห้ามมีข้อมูลสูญหายหรือมาถึงผิดลำดับ

- เป็นการสื่อสารแบบเชื่อมต่อต่อเนื่องตลอดกระบวนการโอนไฟล์ 1 ครั้งไม่ใช้การส่ง request เดี่ยวแล้วจบทันที

Service Model ของ Transport Layer ที่เลือกใช้: TCP

โปรแกรม Hopper เลือกใช้ TCP (Transmission Control Protocol) เป็น transport-layer protocol ด้วยเหตุผลดังด้านล่าง:

- ความน่าเชื่อถือ: การโอนไฟล์ห้ามมีข้อมูลสูญหายระหว่างทาง เพราะเพียงแค่ byte เดียวที่หายไปก็อาจทำให้ไฟล์เสียหายและเปิดใช้งานไม่ได้
- การเรียงลำดับข้อมูล: ข้อมูลไฟล์ต้องมาถึงและถูกประกอบกลับตามลำดับเดิม ซึ่ง TCP รับประกันลำดับของ byte stream ให้โดยอัตโนมัติ
- Flow control และ Congestion control ในตัว: ช่วยควบคุมอัตราการส่งข้อมูลให้เหมาะสมกับความสามารถของผู้รับและสภาพเครือข่ายโดยอัตโนมัติ ซึ่งสำคัญมากเมื่อโอนไฟล์ขนาดใหญ่ผ่านเครือข่ายที่มีความเร็วไม่แน่นอน
- การเชื่อมต่อแบบ connection-oriented: เหมาะกับลักษณะงานที่ต้องเปิด session คู่กันต่อเนื่องตลอดกระบวนการโอนไฟล์มากกว่าการส่ง datagram แยกอิสระ
- เนื่องจากไม่ได้เน้นความเร็วแบบ real-time จึงยอมรับได้สำหรับความหน่วงเล็กน้อยจาก TCP handshake หรือ retransmission

การออกแบบ Application-Layer Protocol: FHP

FHP เป็น text-based protocol ที่ใช้แนวคิดคล้าย HTTP/FTP กล่าวคือแยกส่วน control message (คำสั่งและสถานะ เป็นข้อความ ASCII จบแบบ CRLF) ออกจาก data transfer โดยฝั่งผู้ส่งจะระบุขนาดไฟล์ไว้ใน control message ก่อนเสมอเพื่อให้ฝั่งผู้รับทราบว่าต้องอ่านข้อมูลกี่ byte จึงจะครบ เนื่องจาก TCP เป็น byte stream ที่ไม่มีขอบเขตของข้อความในตัวเอง

1) Request Messages (Client -> Server)

คำสั่ง	รูปแบบ	คำอธิบาย
LIST	LIST\r\n	ขอรายชื่อไฟล์ทั้งหมดบน Server
UPLOAD	UPLOAD <filename> <filesize>\r\n + binary data ขนาด filesize byte	ส่งไฟล์จาก Client ขึ้นไป เก็บ Server
DOWNLOAD	DOWNLOAD <filename>\r\n	ขอไฟล์จาก Server มายัง Client
DELETE	DELETE <filename>\r\n	ขอให้ Server ลบไฟล์ที่ ระบุ
QUIT	QUIT\r\n	ปิดการเชื่อมต่อกับ Server

2) Response Messages (Server -> Client)

รูปแบบทั่วไป: <status code> <status phrase> [metadata]\r\n + binary data

Status code	Status Phrase	เรียกใช้เมื่อ
150	Transfer Starting	เริ่มส่งข้อมูลไฟล์
200	OK	คำสั่งสำเร็จ
226	Transfer Complete	โอนไฟล์เสร็จสมบูรณ์
400	Bad Request	รูปแบบคำสั่งไม่ถูกต้อง
404	File Not Found	ไม่พบไฟล์ที่ร้องขอ
409	File Already Exists	UPLOAD ไฟล์ซ้ำ
500	Server Error	เกิดข้อผิดพลาดฝั่ง Server

3) ตัวอย่างการสื่อสาร (Example Sessions)

C ; Client

S ; Server

ตัวอย่างที่ 1 – UPLOAD

C: UPLOAD report.pdf 204800\r\n

S: 150 Transfer Starting\r\n

C: [ส่งข้อมูล binary ของไฟล์ report.pdf จำนวน 204800 byte]

S: 226 Transfer Complete\r\n

ตัวอย่างที่ 2 – DOWNLOAD

C: DOWNLOAD report.pdf\r\n

S: 150 Transfer Starting 204800\r\n

S: [ส่งข้อมูล binary ของไฟล์ report.pdf จำนวน 204800 byte]

C: (Client รับข้อมูลครบตามขนาดที่ระบุแล้ว ปิดการรับข้อมูล)

ตัวอย่างที่ 3 – LIST

C: LIST\r\n

S: 200 OK 3\r\n

S: report.pdf 204800\r\n

S: photo.jpg 51200\r\n

S: notes.txt 1024\r\n

ตัวอย่างที่ 4 – DOWNLOAD ไฟล์ที่ไม่มีอยู่บน Server (error case)

C: DOWNLOAD missing.txt\r\n

S: 404 File Not Found\r\n